Buffer: Working with Binary Data

10 min read · Updated 24 Jul 2026 · Topic 23 of 70

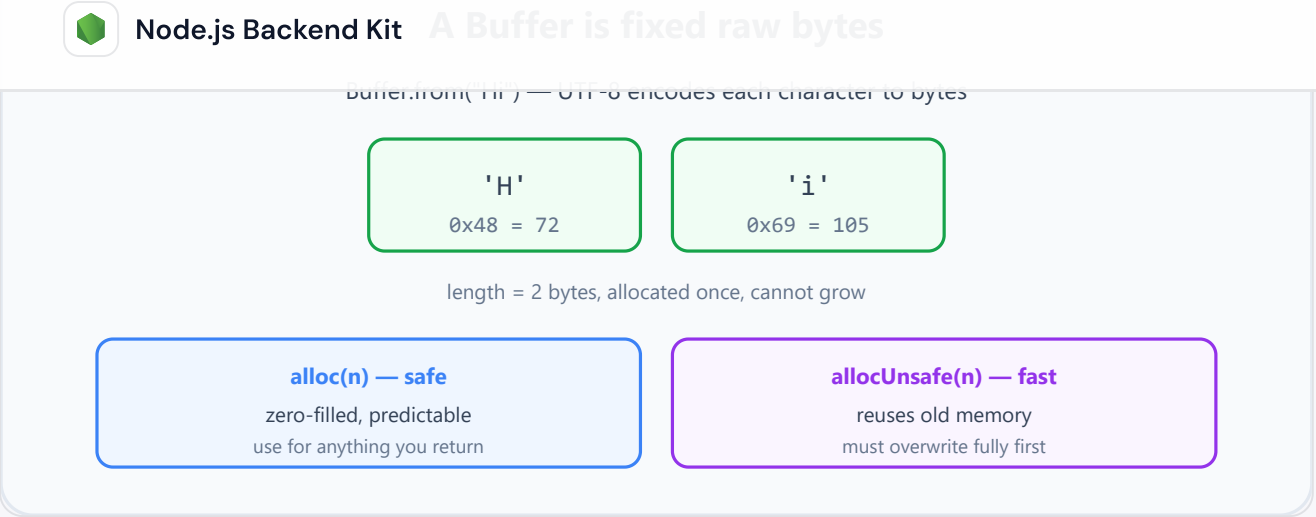
Why it matters

JavaScript grew up in the browser handling text and objects, not raw bytes off a socket or disk. The server needs both. Every file you read, every TCP packet, every image upload, every cryptographic digest is fundamentally a sequence of bytes, and Node represents those bytes with the `Buffer` class. If you have ever wondered why `fs.readFile` without an encoding returns something that prints as `<Buffer 48 69>`, or why a multi-byte emoji breaks when you slice a chunk in half, the answer is buffers.

Understanding `Buffer` is what lets you handle uploads, parse binary protocols, convert between encodings, and reason about memory correctly. It is also a favourite interview topic because it exposes whether you understand the boundary between *bytes* and *characters* — a distinction that, when blurred, produces corrupted data, wrong lengths, and subtle security bugs. Buffers are where "it's just a string" stops being true.

The concept, simply

A `Buffer` is a fixed-length chunk of raw memory holding bytes, each an integer from 0 to 255. It is Node's original binary type, built on top of the standard `Uint8Array` (in fact a `Buffer` *is* a `Uint8Array` with extra convenience methods). Unlike a JavaScript string, a buffer is not text and has no inherent character meaning — it is just bytes. To turn bytes into readable text, or text into bytes, you apply an **encoding** (`utf8`, `hex`, `base64`, `ascii`, `latin1`). The same bytes look completely different depending on which encoding you use to view them.



A Buffer is fixed raw bytes. alloc zero-fills for safety; allocUnsafe is faster but hands you old memory you must overwrite.

How it works (deep)

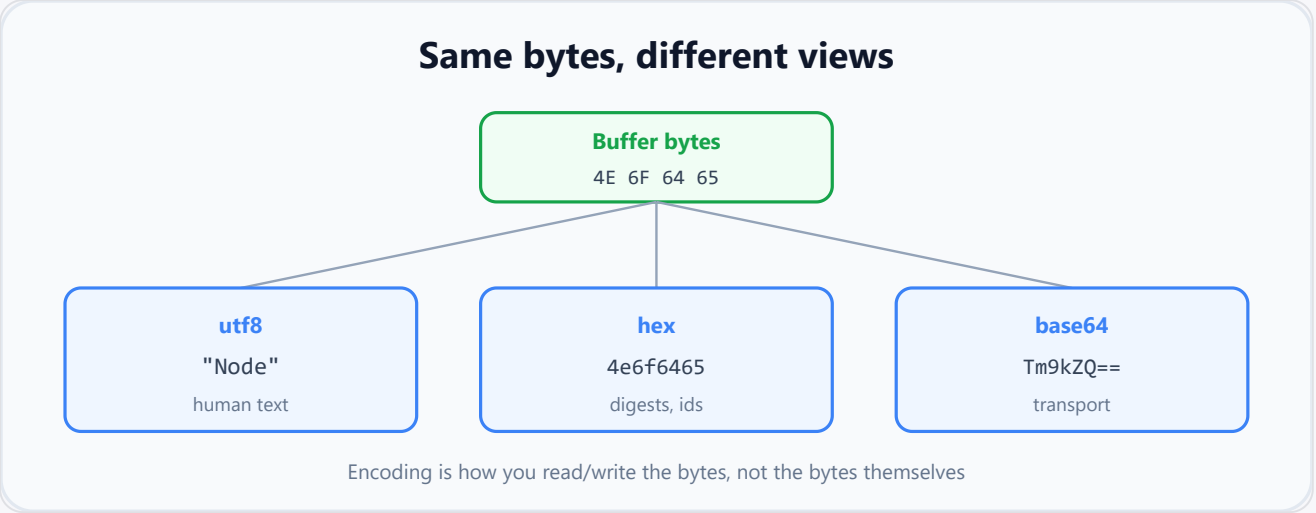
Buffers live in memory allocated outside the main V8 JavaScript heap. This matters: it keeps large binary blobs from bloating the heap and pressuring the garbage collector, and it allows fast, zero-copy handoff to C++ and the OS during I/O. A buffer is backed by an `ArrayBuffer` — the raw memory — and the `Buffer` instance is a typed view over it.

The crucial property is that a buffer has a **fixed length set at allocation**. You cannot grow it. If you need more room you allocate a new, larger buffer and copy, or you collect chunks in an array and `Buffer.concat` them at the end. This is the opposite of a JavaScript string or array that grows on demand, and it is why streaming code accumulates chunks and joins once rather than appending repeatedly.

Allocation comes in two forms with a real safety difference. `Buffer.alloc(n)` returns `n` bytes **zero-filled** — safe and predictable. `Buffer.allocUnsafe(n)` is faster because it hands you a slice of previously used memory *without clearing it*, which means it may contain leftover data from earlier operations — potentially another request's secrets. It is safe only if you immediately overwrite every byte before reading or returning it. Use `alloc` by default; reach for `allocUnsafe` only in hot paths where you provably fill the whole buffer.

Encoding is where bytes meet characters, and it is the number-one source of buffer bugs. In **UTF-8**, ASCII characters take one byte but many characters take two, three, or

stream at an arbitrary boundary — as happens between stream chunks — you can cut a multi-byte character in half, and decoding each half independently produces garbage. The fix is `string_decoder`'s `StringDecoder`, which buffers incomplete multi-byte sequences across chunk boundaries.



One buffer, three encodings. Encoding is how you interpret bytes for reading or writing — it does not change the underlying bytes.

You typically encounter buffers whenever data crosses the process boundary as bytes: `fs` reads without an encoding, network sockets, `crypto` digests and random bytes, `zlib` compression, image and file processing, and any binary protocol. Text-first APIs hide buffers behind an encoding argument; drop the encoding and the buffer surfaces.

It is worth being precise about what each **encoding label** actually means, because they are not interchangeable views of the same idea. `'utf8'` is the real text codec — variable-width, 1–4 bytes per character, and the correct default for anything human-readable. `'hex'` and `'base64'` are *transport* encodings: they render arbitrary bytes as safe ASCII (2 hex chars or ~1.33 base64 chars per byte) so binary can ride through JSON, URLs, or headers — Node also accepts `'base64url'`, the URL-safe variant. `'latin1'` (a.k.a. `'binary'`) is a 1:1 byte-to-code-point map that never fails but silently corrupts real Unicode. `'ascii'` is 7-bit only and masks the high bit off every byte, so it mangles anything above `0x7F`. Choosing `'ascii'` or `'latin1'` for what is genuinely UTF-8 data is a common, quiet source of mojibake.

`ArrayBuffer` — no copy — whereas `Buffer.from(string)` and `Buffer.from(array)` allocate fresh memory. Second, small buffers are quietly pooled: for allocations under `Buffer.poolSize >> 1` (4 KB by default), `Buffer.allocUnsafe` and `Buffer.from` carve a slice out of a shared internal `ArrayBuffer`. That is why `buf.byteOffset` is often non-zero and why `buf.buffer` can be far larger than `buf.length` — the `Buffer` is a window onto a bigger pool, not a private allocation. If you hand `buf.buffer` to another API expecting the whole thing, you leak neighbouring buffers' bytes; always pass `buf.byteOffset` and `buf.byteLength` too.

Code examples

Creating buffers and converting between bytes and encodings:

JAVASCRIPT Copy

```
// From text (defaults to utf8):
const buf = Buffer.from('Node');
buf;                          //
buf.length;                   // 4 bytes
buf.toString('hex');          // '4e6f6465'
buf.toString('base64');       // 'Tm9kZQ=='
buf.toString('utf8');         // 'Node'

// Decode data that arrived in a different encoding:
const fromB64 = Buffer.from('Tm9kZQ==', 'base64');
fromB64.toString('utf8');  // 'Node'

// Safe vs fast allocation:
const zeroed = Buffer.alloc(8);         //
const fast   = Buffer.allocUnsafe(8);  // arbitrary leftover bytes — must
fast.fill(0);                           // now safe to use
```

Why byte length and character length differ — the classic emoji trap:

JAVASCRIPT Copy

```
Buffer.byteLength(s, 'utf8');   // 5  (1 byte + 4 bytes)

// Slicing bytes blindly can cut a character in half:
const b = Buffer.from(s, 'utf8');
b.subarray(0, 2).toString('utf8'); // 'a\uFFFD' — broken: half an emoji
```

Collecting stream chunks the right way — accumulate, then concat once:

JAVASCRIPT Copy

```
import { createReadStream } from 'node:fs';

async function readAllBytes(path) {
  const chunks = [];
  for await (const chunk of createReadStream(path)) {
    chunks.push(chunk); // each chunk is a Buffer
  }
  return Buffer.concat(chunks); // single allocation at the end
}

// For text streams, decode across chunk boundaries safely:
import { StringDecoder } from 'node:string_decoder';
const decoder = new StringDecoder('utf8');
// decoder.write(chunk) holds back any partial multi-byte char until compl
```

Reading and writing fixed-width binary fields (parsing a simple protocol header):

JAVASCRIPT Copy

```
// Build a 4-byte big-endian header: version(1) + payloadLength(uint16) +
const header = Buffer.alloc(4);
header.writeUInt8(2, 0);          // version = 2 at offset 0
header.writeUInt16BE(512, 1);     // length  = 512 at offset 1 (2 bytes)
header.writeUInt8(1, 3);          // flag    = 1 at offset 3

// Read it back:
```

Common mistakes & gotchas

- **Confusing `str.length` with byte length.** `str.length` counts UTF-16 code units; bytes on the wire are `Buffer.byteLength(str, 'utf8')`. They differ for any non-ASCII text, which breaks `Content-Length` headers and offset math.
- **Slicing a multi-byte character in half.** Splitting UTF-8 bytes at a chunk boundary and decoding each half yields replacement characters (garbage). Use `StringDecoder` for streamed text.
- **Using `allocUnsafe` without fully overwriting it.** It exposes leftover memory — possibly another request's data — which is both a correctness and a security bug. Default to `alloc`.
- **Trusting the deprecated `new Buffer()` constructor.** `new Buffer(number)` historically returned uninitialised memory. It is deprecated; always use `Buffer.alloc` / `Buffer.from`.
- **Building a big buffer by repeated `Buffer.concat` in a loop.** Buffers cannot grow, so each concat reallocates and copies — O(n²). Push chunks into an array and `Buffer.concat` once.
- **Assuming `buf.slice()` copies.** Both `slice` and `subarray` return a *view* sharing the same memory — mutating one changes the other. Use `Buffer.from(buf)` for an independent copy.
- **Forgetting endianness in binary protocols.** `readUInt16BE` vs `readUInt16LE` read the same bytes as different numbers. Match the protocol's byte order exactly.
- **Passing `buf.buffer` to another API without the offset and length.** Because small buffers are pooled, `buf.buffer` is frequently a shared 8 KB `ArrayBuffer`, not your bytes alone. Handing the bare `ArrayBuffer` to `new Uint8Array(buf.buffer)` or a crypto call exposes unrelated buffers' data. Always pass `buf.byteOffset` and `buf.byteLength`, or copy with `Buffer.from(buf)`.
- **Reading multi-byte integers that exceed 48 bits with the `*BE` / `*LE` helpers.** `readUInt32BE` is fine, but JavaScript numbers lose precision past 2^53, so 64-bit

- **Picking `'ascii'` or `'latin1'` for text that is actually UTF-8.** Both decode byte-per-character and never throw, so the bug is invisible until a non-ASCII character arrives and turns into mojibake. Use `'utf8'` for text and a transport encoding (`'hex'` / `'base64'`) for raw bytes.

Best practices

- **Use `Buffer.alloc` and `Buffer.from`**; never the deprecated `new Buffer()` constructor.
- **Default to `alloc` (zero-filled);** use `allocUnsafe` only in hot paths where you overwrite every byte.
- **Measure wire size with `Buffer.byteLength(str, 'utf8')`**, not `str.length`.
- **Decode streamed text with `StringDecoder`** so multi-byte characters survive chunk boundaries.
- **Accumulate chunks in an array and `Buffer.concat` once** rather than concatenating repeatedly.
- **Copy with `Buffer.from(buf)` when you need independence;** remember `slice` / `subarray` share memory.
- **Be explicit about encoding and endianness** at every byte boundary — never let it default by accident.

⭐ Interview lens

> **Q: What is a Buffer and how does it differ from a string?** A `Buffer` is a fixed-length region of raw bytes (0–255 each), stored outside the V8 heap and backed by an `ArrayBuffer`; it is a `Uint8Array` subclass. A string is text made of UTF-16 code units with character meaning. Buffers are bytes; strings are characters. You convert between them with an explicit encoding.

previously used memory, which may still hold sensitive data from earlier operations. If you read it before overwriting, you can leak that data. It is only safe when you immediately fill every byte, and it exists purely for performance.

Q: `'caf\u00e9'.length` is 4, but how many bytes is it in UTF-8? Five — the accented e is two bytes in UTF-8 while the other three are one each. This is why you must use `Buffer.byteLength` for anything that counts wire bytes, like `Content-Length`.

Q: You read a stream in chunks and decode each with `chunk.toString()`; some characters are garbled. Why? A multi-byte UTF-8 character was split across two chunks, so each half decoded to a replacement character. Fix it with `StringDecoder`, which holds incomplete byte sequences until the next chunk completes them.

Q: Does `buf.slice(0, 4)` copy the data? No. `slice` (and `subarray`) return a view over the same underlying memory, so mutating the slice mutates the original. Use `Buffer.from(buf.subarray(0,4))` for a real copy.

Q: Why are buffers stored off the V8 heap? To avoid bloating the JavaScript heap with large binary blobs and pressuring the garbage collector, and to allow zero-copy handoff to native code and the OS during I/O. It keeps binary-heavy workloads efficient.

Cheat-card

- `Buffer` = fixed-length raw bytes (0–255), a `Uint8Array` subclass, off the V8 heap.

- Convert: `buf.toString('hex'|'base64'|'utf8')`; `Buffer.from(s, 'base64')`.
- Bytes on the wire = `Buffer.byteLength(str,'utf8')`, not `str.length`.
- Streamed text → `StringDecoder` to survive chunk splits.
- Many chunks → array + one `Buffer.concat` (buffers can't grow).
- `slice` / `subarray` share memory; `Buffer.from(buf)` copies.
- Binary fields: `readUInt16BE` / `LE` — mind endianness.
- Never use deprecated `new Buffer()`.

QUICK CHECK — no marks, just for you.

1. Why is Buffer.allocUnsafe(n) called "unsafe"?

- ◯ It can allocate more than the available RAM and crash the process
- ◯ It returns a read-only buffer that throws if you try to write to it
- ◯ It skips zero-filling and hands you a slice of previously used memory that may still hold sensitive data until you overwrite it
- ◯ It allocates on the V8 heap, pressuring the garbage collector

Check answer

2. The string "a" followed by a 4-byte emoji has .length of 3. How many bytes is it in UTF-8, and why does that matter?

- ◯ 3 bytes; str.length always equals the UTF-8 byte length
- ◯ 5 bytes, but the difference never affects headers like Content-Length
- ◯ 8 bytes, because every character is padded to two bytes in UTF-8

Check answer

3. Does buf.slice(0, 4) return an independent copy of the data?

- [ ] No; slice and subarray return a view sharing the same underlying memory, so mutating one mutates the other
- [ ] Yes; slice always allocates a fresh, independent buffer
- [ ] Yes, but only when the buffer is larger than 4 KB
- [ ] No; slice throws unless you pass a third "copy" argument

Check answer

‹ Previous
path, os & URL: Portable Filesystem and Environment Info

Next ›
crypto: Hashing, HMAC, Random & Encryption

LEARN MORE

ARTICLES & DOCS

Buffer | Node.js Documentation
nodejs.org

Uint8Array - JavaScript | MDN
developer.mozilla.org

VIDEOS

Node.js Backend Kit

Hand-picked references for this lesson.